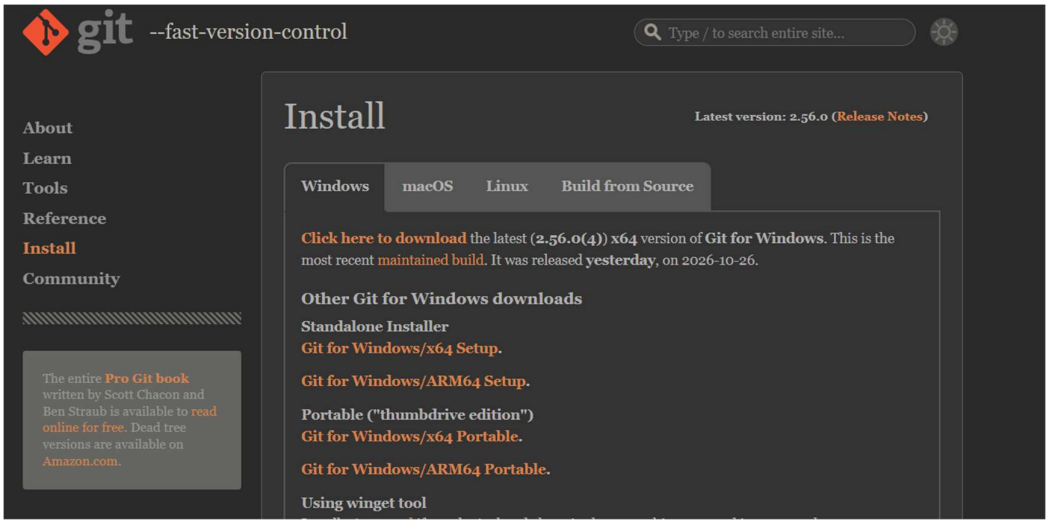
Git Task 2

1. Install git.

Open git in website and download the installer. After installing open Git to check if it is working or not.



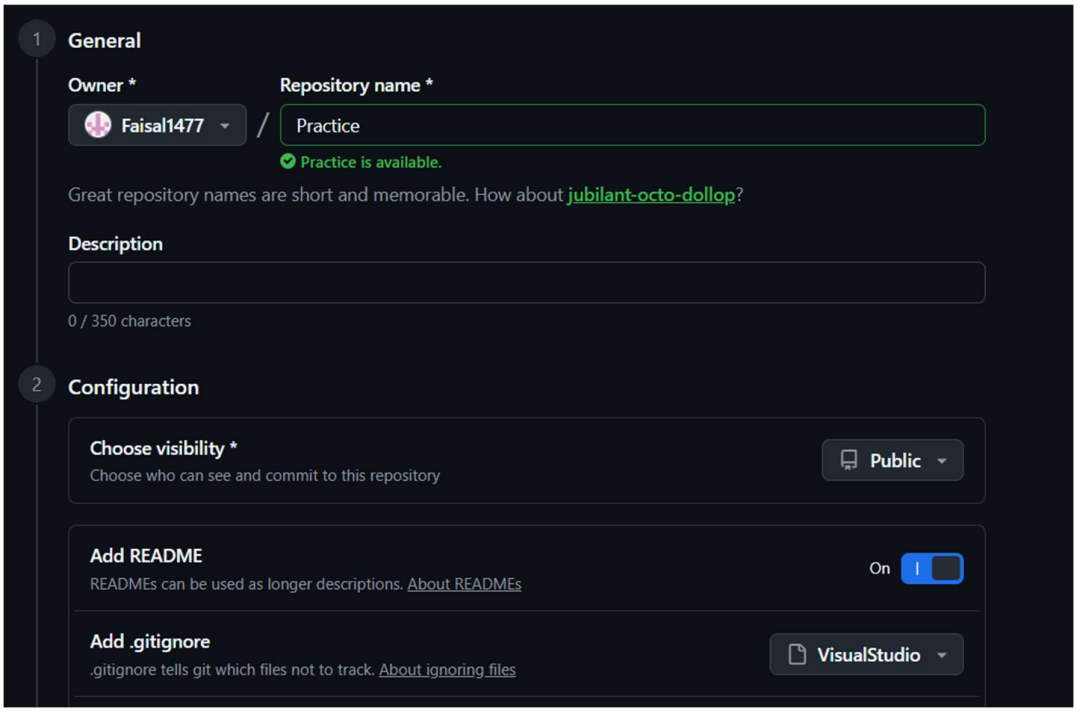

```
Faisal@LAPTOP-4DO8SS4E MINGW64 ~
$ git --version
git version 2.37.1.windows.1
```

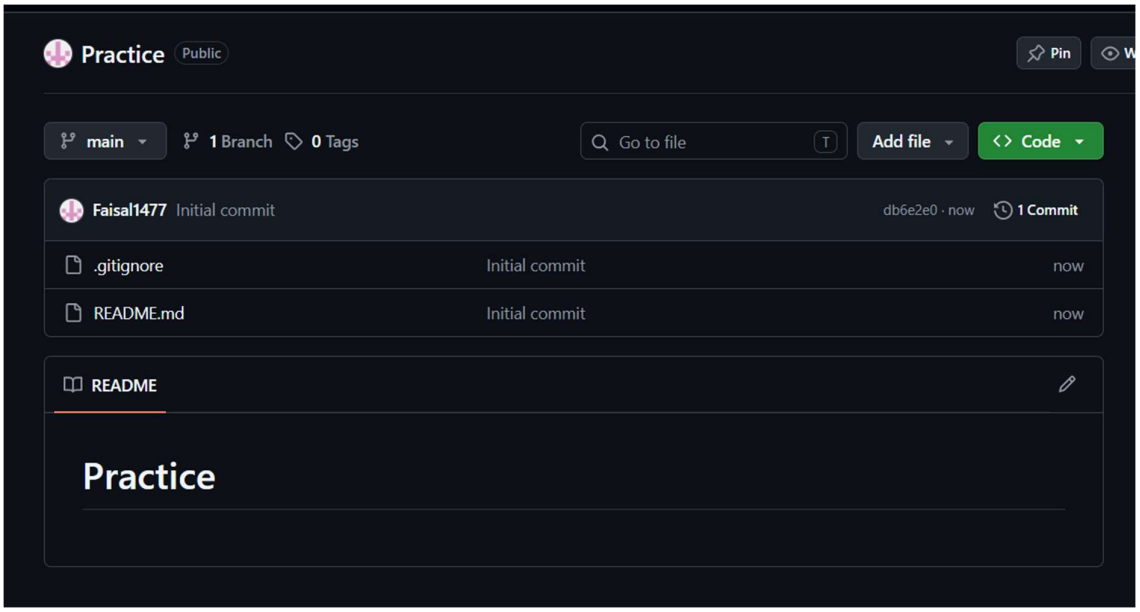
2. Create a repo in github with README.md and .ignore file.

Step 1:- Open GitHub and create new repository.

Step 2:- After naming the repo scroll down and enable readme

Step 3:- Also select .gitignore for the project according then create repo





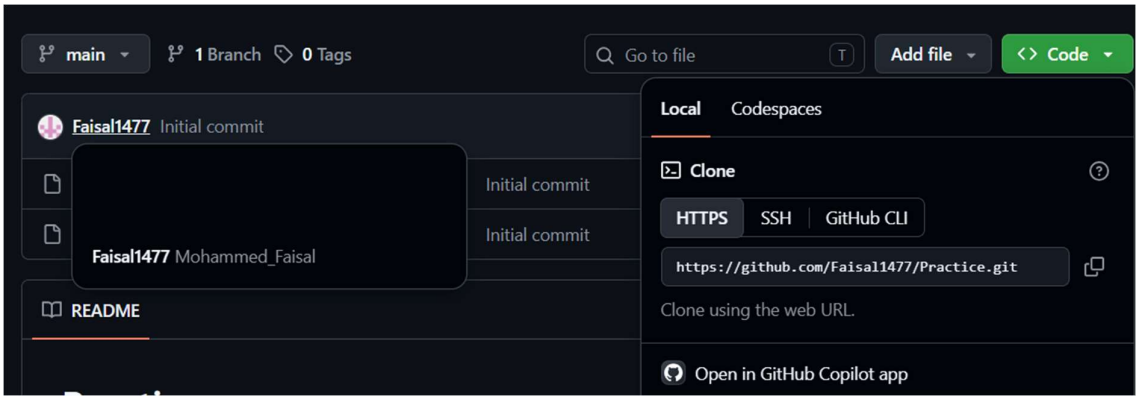
3. Clone the created repo to local.

Step 1:-Open a repo in GitHub

Step 2:- Take the link from code field in the repo

Step 3:- Then use clone command in any directory to clone the repo

Step 4:- Entire repo will be cloned and a directory will be created with repo name




```
Faisal@LAPTOP-4DO8SS4E MINGW64 ~/Downloads/Day 8 (master)
$ ls
 10-Git-and-Github.mp4   git-2.1.png   git-3.1.png       '~WRL3640.tmp'
'Git Task 2.docx'        git-2.2.png  '~$t Task 2.docx'

Faisal@LAPTOP-4DO8SS4E MINGW64 ~/Downloads/Day 8 (master)
$ git clone https://github.com/Faisal1477/Practice.git
Cloning into 'Practice'...
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (4/4), done.

Faisal@LAPTOP-4DO8SS4E MINGW64 ~/Downloads/Day 8 (master)
$ ls
 10-Git-and-Github.mp4   Practice/     git-2.2.png  '~$t Task 2.docx'
'Git Task 2.docx'        git-2.1.png   git-3.1.png  '~WRL3640.tmp'
```

4. Create two files in local repo.

Step 1:- Use touch to create two files

Step 2:- Then use git add to track the files

```
Faisal@LAPTOP-4DO8SS4E MINGW64 ~/Downloads/Day 8/Practice (main)
$ ls
README.md

Faisal@LAPTOP-4DO8SS4E MINGW64 ~/Downloads/Day 8/Practice (main)
$ touch text1 text2

Faisal@LAPTOP-4DO8SS4E MINGW64 ~/Downloads/Day 8/Practice (main)
$ ls
README.md  text1  text2
```

```
Faisal@LAPTOP-4DO8SS4E MINGW64 ~/Downloads/Day 8/Practice (main)
$ git add .

Faisal@LAPTOP-4DO8SS4E MINGW64 ~/Downloads/Day 8/Practice (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   text1
        new file:   text2
```

5. Commit two files and push to central Repository.

Step 1:- First check the status for any changes

Step 2:- if the files are untracked use git add

Step 3:- Then use git commit to commit the changes

```
Faisal@LAPTOP-4DO8SS4E MINGW64 ~/Downloads/Day 8/Practice (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   text1
        new file:   text2


Faisal@LAPTOP-4DO8SS4E MINGW64 ~/Downloads/Day 8/Practice (main)
$ git commit -m "Two new files are added"
[main 2800a99] Two new files are added
 2 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 text1
 create mode 100644 text2

Faisal@LAPTOP-4DO8SS4E MINGW64 ~/Downloads/Day 8/Practice (main)
$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
```

```
Faisal@LAPTOP-4DO8SS4E MINGW64 ~/Downloads/Day 8/Practice (sample)
$ git push -u origin main
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/Faisal1477/Practice.git
   db6e2e0..2800a99  main -> main
branch 'main' set up to track 'origin/main'.
```

6. Create a branch in local and create a sample file and push to central.

Step 1:- open a git initialized folder

Step 2:- Check the branches available with git branch –list

Step 3:- Now to create a new branch use git branch branch_name

Step 4:- Then to switch to other branch use git checkout branch_name

Step 5:- Create sample file in the new branch. Then add the files to tracking list. Commit the changes done

Step 6:-Push it to the central repo with git push -u origin new_branch

```
Faisal@LAPTOP-4DO8SS4E MINGW64 ~/Downloads/Day 8/Practice (main)
$ git branch sample

Faisal@LAPTOP-4DO8SS4E MINGW64 ~/Downloads/Day 8/Practice (main)
$ git branch --list
* main
  sample

Faisal@LAPTOP-4DO8SS4E MINGW64 ~/Downloads/Day 8/Practice (main)
$ git checkout sample
Switched to branch 'sample'
```

```
Faisal@LAPTOP-4DO8SS4E MINGW64 ~/Downloads/Day 8/Practice (sample)
$ touch sampletxt

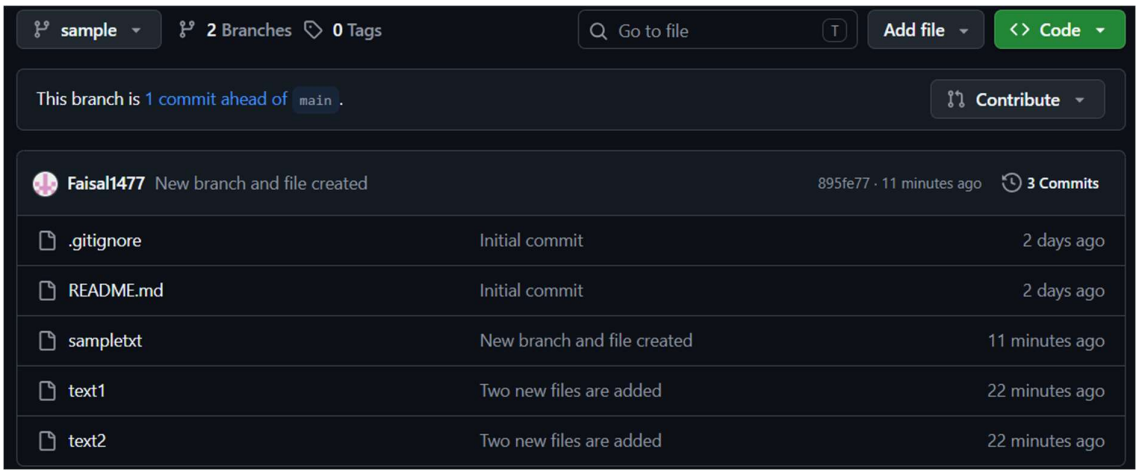
Faisal@LAPTOP-4DO8SS4E MINGW64 ~/Downloads/Day 8/Practice (sample)
$ ls
README.md  sampletxt  text1  text2

Faisal@LAPTOP-4DO8SS4E MINGW64 ~/Downloads/Day 8/Practice (sample)
$ git add .

Faisal@LAPTOP-4DO8SS4E MINGW64 ~/Downloads/Day 8/Practice (sample)
$ git status
On branch sample
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   sampletxt


Faisal@LAPTOP-4DO8SS4E MINGW64 ~/Downloads/Day 8/Practice (sample)
$ git commit -m "New branch and file created"
[sample 895fe77] New branch and file created
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 sampletxt
```

```
Faisal@LAPTOP-4DO8SS4E MINGW64 ~/Downloads/Day 8/Practice (sample)
$ git push -u origin sample
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 12 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 518 bytes | 518.00 KiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), done.
remote:
remote: Create a pull request for 'sample' on GitHub by visiting:
remote:      https://github.com/Faisal1477/Practice/pull/new/sample
remote:
To https://github.com/Faisal1477/Practice.git
 * [new branch]      sample -> sample
branch 'sample' set up to track 'origin/sample'.
```



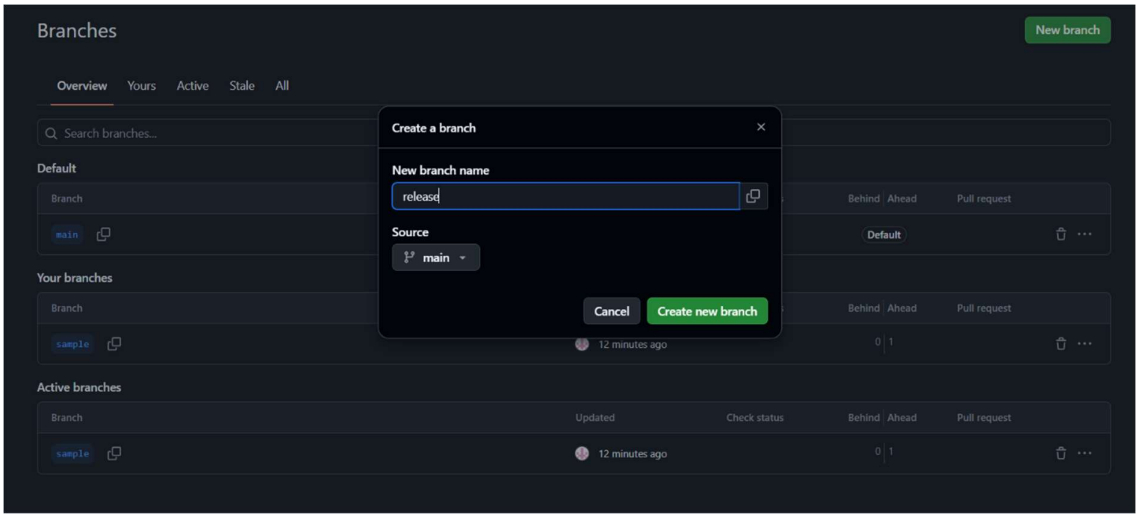
7. Create a branch in github and clone that to local.

Step 1:- Open the GitHub and Click on branches to create new branch

Step 2:- Go to project folder of that repo in local machine

Step 3:- Use git fetch origin to get the updates from Github

Step 4:- Now we can switch to new_branch



```
Faisal@LAPTOP-4DO8SS4E MINGW64 ~/Downloads/Day 8/Practice (sample)
$ git fetch
From https://github.com/Faisal1477/Practice
 * [new branch]      release    -> origin/release
```

```
Faisal@LAPTOP-4DO8SS4E MINGW64 ~/Downloads/Day 8/Practice (sample)
$ git switch --track origin/release
Switched to a new branch 'release'
branch 'release' set up to track 'origin/release'.

Faisal@LAPTOP-4DO8SS4E MINGW64 ~/Downloads/Day 8/Practice (release)
$ git branch
  main
* release
  sample
```

8. Merge the created branch with master in git local.

Step 1:- Check whether there are multiple branches in the project or not with git branch --list command

Step 2:- After checking go to main branch where we want to merge.

Step 3:- Then use git merge branch_name to merge into current branch

```
Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (main)
$ git switch release
Switched to branch 'release'
A       Addfile

Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (release)
$ git add .
warning: in the working copy of 'text.txt', LF will be replaced by CRLF the next time Git touches it

Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (release)
$ git commit -m "New file added in release branch"
[release 5d6501d] New file added in release branch
 2 files changed, 1 insertion(+)
 create mode 100644 Addfile
 create mode 100644 text.txt

Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (release)
$ git switch main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 3 commits.
  (use "git push" to publish your local commits)

Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (main)
$ git merge release
Updating 65d7113..5d6501d
Fast-forward
 Addfile  | 0
 text.txt | 1 +
 2 files changed, 1 insertion(+)
 create mode 100644 Addfile
 create mode 100644 text.txt
```

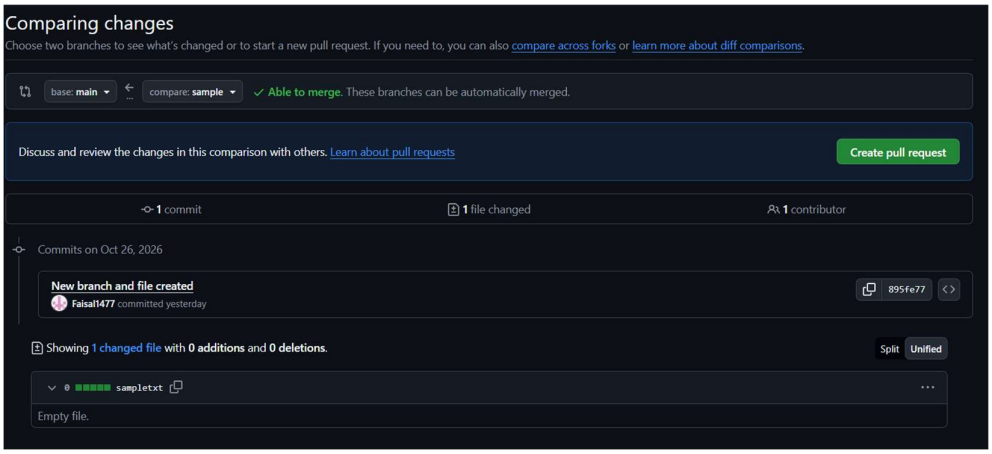
9. Merge the created branch with master in github by sending a pull request.

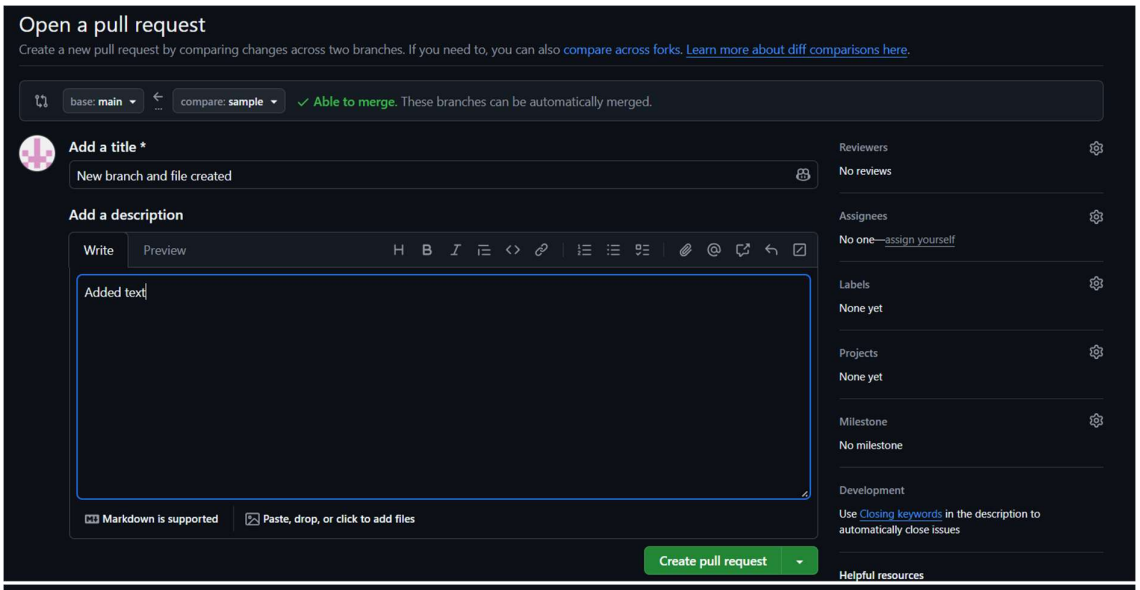
Step 1:- Go to GitHub and open the repo. There we can see compare and pull request button click on it.

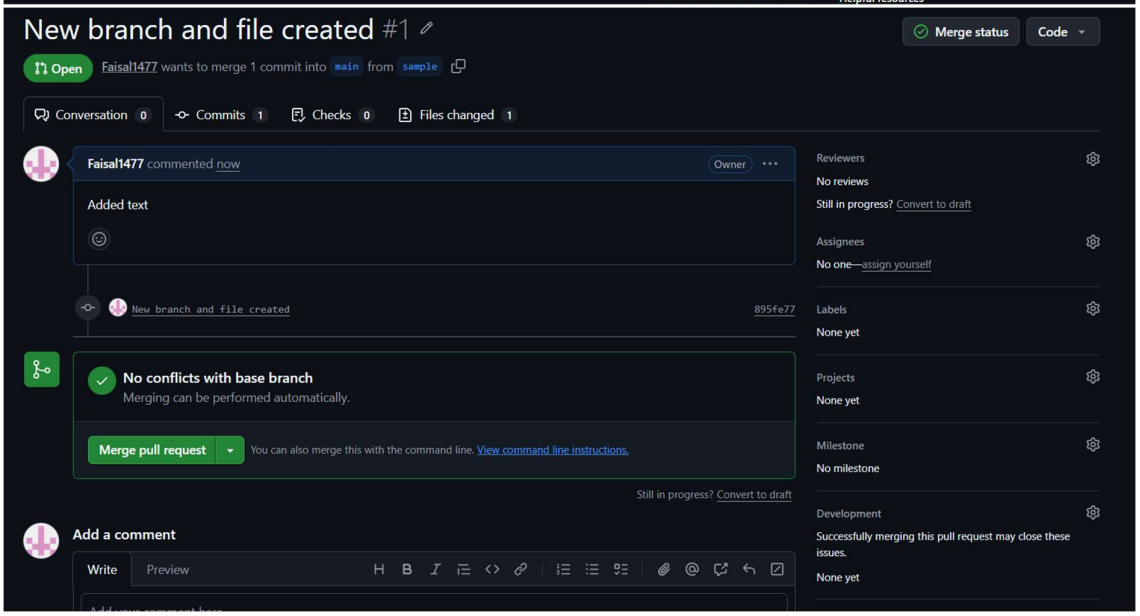
Step 2:- After clicking this we have to select which branch should be base and the source branch

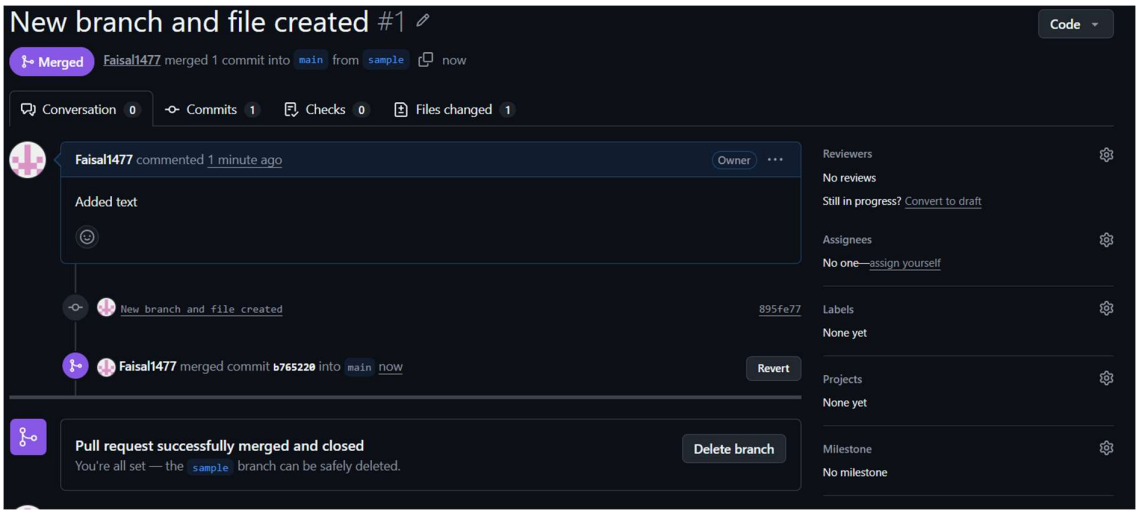
Step 3:- If the branch has difference we can merge. Then create Pull request

Step 4:- If there are no merge conflicts then merge is possible









10. Create a file in local and send that to branch in github.

Step 1:- Create a file in new_branch. Add it to tracking and commit the changes.

Step 2:- Now while using Push command specify the branch where we want to push the files

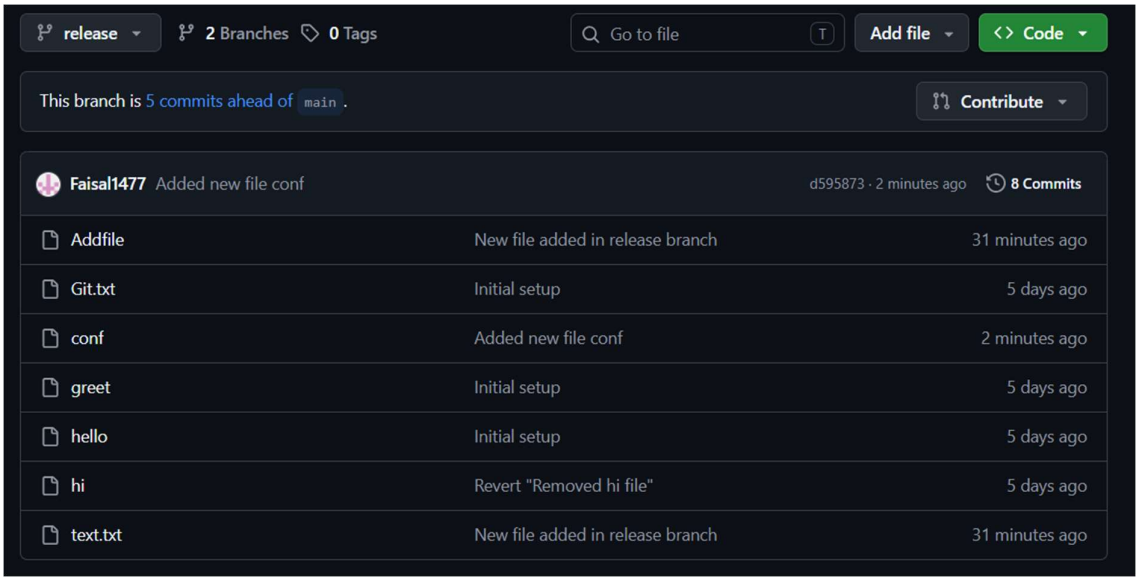
```
Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (main)
$ git switch release
Switched to branch 'release'

Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (release)
$ echo "This file was created in release branch" > conf

Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (release)
$ git add conf
warning: in the working copy of 'conf', LF will be repla

Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (release)
$ git commit -m "Added new file conf"
[release d595873] Added new file conf
 1 file changed, 1 insertion(+)
 create mode 100644 conf
```

```
Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (release)
$ git push -u origin release
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 12 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (11/11), 1.28 KiB | 654.00 KiB/s, done.
Total 11 (delta 3), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (3/3), completed with 1 local object.
To https://github.com/Faisal1477/Git-practice.git
   aeb6302..d595873  release -> release
branch 'release' set up to track 'origin/release'.
```



11. Clone only a branch from github to local.

Step 1:- Open the GitHub and Click on code to copy the repo Url

Step 2:- Open any folder in the local machine. Then use Git clone -b branch_name url to clone the project into local repo

Step 3:- Then the project is cloned in the folder with repo name

```
Faisal@LAPTOP-4DO8SS4E MINGW64 /d/clone
$ git clone -b release https://github.com/Faisal1477/Git-practice.git
Cloning into 'Git-practice'...
remote: Enumerating objects: 19, done.
remote: Counting objects: 100% (19/19), done.
remote: Compressing objects: 100% (10/10), done.
remote: Total 19 (delta 5), reused 16 (delta 4), pack-reused 0 (from 0)
Receiving objects: 100% (19/19), done.
Resolving deltas: 100% (5/5), done.

Faisal@LAPTOP-4DO8SS4E MINGW64 /d/clone
$ ls
Git-practice/

Faisal@LAPTOP-4DO8SS4E MINGW64 /d/clone
$ cd Git-practice/

Faisal@LAPTOP-4DO8SS4E MINGW64 /d/clone/Git-practice (release)
$ ls
Addfile  Git.txt  conf  greet  hello  hi  text.txt
```

12. Create a file with all passwords and make that untrackable with git.

Step 1:- Create a file with all the passwords.

Step 2:- Without doing git add create a .gitignore file and add the password file into it.

Step 3:- Then the file with password will become untrackable and only .gitignore will be available

Step 4:- Now track the gitignore file and commit changes. Then finally push the files into repo to see If is was successful or not

```
Faisal@LAPTOP-4DO8SS4E MINGW64 /d/clone/Git-practice (release)
$ ls
Addfile  Git.txt  conf  greet  hello  hi  text.txt

Faisal@LAPTOP-4DO8SS4E MINGW64 /d/clone/Git-practice (release)
$ touch passwords

Faisal@LAPTOP-4DO8SS4E MINGW64 /d/clone/Git-practice (release)
$ vi .gitignore
```

```
MINGW64:/d/clone/Git-practice
Passwords
~
~
~
~
~
~
```

```
Faisal@LAPTOP-4DO8SS4E MINGW64 /d/clone/Git-practice (release)
$ git add .
warning: in the working copy of '.gitignore', LF will be replaced by CRLF the next time Git touches it

Faisal@LAPTOP-4DO8SS4E MINGW64 /d/clone/Git-practice (release)
$ git status
On branch release
Your branch is up to date with 'origin/release'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   .gitignore
```

13. Make a commit and make that commit reset without savings changes.

Step 1:- Add some files into the project folder then commit the changes.

Step 2:- Then we can use git reset with –hard flag to remove everything from that commit and HEAD~1 to reset back one commit

Step 3:- After using this everything from previous commit will be removed

```
Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Tasks (main)
$ touch hello

Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Tasks (main)
$ git add .

Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Tasks (main)
$ git commit -m "Added Hello File"
[main d2a6753] Added Hello File
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 hello

Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Tasks (main)
$ git log
commit d2a6753630eeb8f814c52e14b3e9bb527522cebc (HEAD -> main)
Author: Faisal1477 <mohdfaisal1477@gmail.com>
Date:   Tue Oct 27 18:50:59 2026 +0530

    Added Hello File
```

```
Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Tasks (main)
$ git reset --hard HEAD~1
HEAD is now at 9a4cb02 Readme file is added &

Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Tasks (main)
$ git log
commit 9a4cb024627c9d748f2d392e3e3b0c4a02268912 (HEAD -> main)
Author: Faisal1477 <mohdfaisal1477@gmail.com>
Date:   Tue Oct 27 18:05:06 2026 +0530

    Readme file is added &

    Dummy file added

commit fc96ab13f06993de374a55ce24cd0fb8455ecb46
Author: Faisal1477 <mohdfaisal1477@gmail.com>
Date:   Sat Oct 24 12:40:09 2026 +0530

    Linux, Bash, Git task documents uploaded
```

14. Revert a commited commit to the older version.

Step 1:- First create multiple commits for this to work

Step 2:- Now to get the commit id use git log

Step 3:- Copy first 5 characters from this id.

Step 4:- Use git revert id command to revert back that commit.

```
Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (main)
$ git log --oneline
78b2d9b (HEAD -> main) Merge branch 'main' of https://github.com/Faisal1477/Git-practice
aeb6302 (origin/main) New file created
28aaf5c Revert "Removed hi file"
caddaf9 Removed hi file
6772b06 Initial setup

Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (main)
$ git revert aeb6302
[main 65d7113] Revert "New file created"
 1 file changed, 1 deletion(-)
 delete mode 100644 Text

Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (main)
$ ll
total 1
-rw-r--r-- 1 Faisal 197609  0 Oct 22 17:29 Addfile
-rw-r--r-- 1 Faisal 197609 11 Oct 22 16:16 Git.txt
-rw-r--r-- 1 Faisal 197609  0 Oct 22 16:09 greet
-rw-r--r-- 1 Faisal 197609  0 Oct 22 16:09 hello
-rw-r--r-- 1 Faisal 197609  0 Oct 22 17:19 hi
```

```
MINGW64:/d/Git
Revert "New file created"

This reverts commit aeb6302681285f03469e7f1ccaaae1c417eb5249.

# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# On branch main
# Your branch is ahead of 'origin/main' by 2 commits.
#   (use "git push" to publish your local commits)
#
# Changes to be committed:
#       deleted:    Text
#
# Untracked files:
#       Addfile
#
```

15. Push a file to stash without savings the changes and work on another file.

Step 1:- First create a new file. Then add it to tracking.

Step 2:- Now we can use git stash to hide this file from being tracked.

Step 3:- This will move the file to stash. To check the stash use git stash list

```
Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (main)
$ git status
On branch main
Your branch is ahead of 'origin/main' by 3 commits.
  (use "git push" to publish your local commits)

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        Addfile

nothing added to commit but untracked files present (use "git add" to track)

Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (main)
$ git add .
```

```
Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (main)
$ git stash
Saved working directory and index state WIP on main: 9c690de Addfile is added
```

```
Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (main)
$ git stash list
stash@{0}: WIP on main: 65d7113 Revert "New file created"
stash@{1}: WIP on main: 9c690de Addfile is added
```

16. Undo the stash file and start working on that again.

Step 1:- Check the git Stash List

Step 2:- If there is only one stash in the list use git stash pop. We can also pop with stash Id.

Step 3:- Now we can start working on the file

```
Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (main)
$ git stash list
stash@{0}: WIP on main: 65d7113 Revert "New file created"
stash@{1}: WIP on main: 9c690de Addfile is added
```

```
Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Git (main)
$ git stash pop
On branch main
Your branch is ahead of 'origin/main' by 3 commits.
  (use "git push" to publish your local commits)

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   Addfile

Dropped refs/stash@{0} (8448f277280013776d9eee79eaa13f4731c2dea6)
```

17. Generate a ssh-keygen and configure into github.

Step 1:- First go to Gitbash and use ssh-keygen command to generate new ssh key

Step 2:- It will ask us to enter the directory path where the ssh key file should be saved. If entered without any path it will take default path

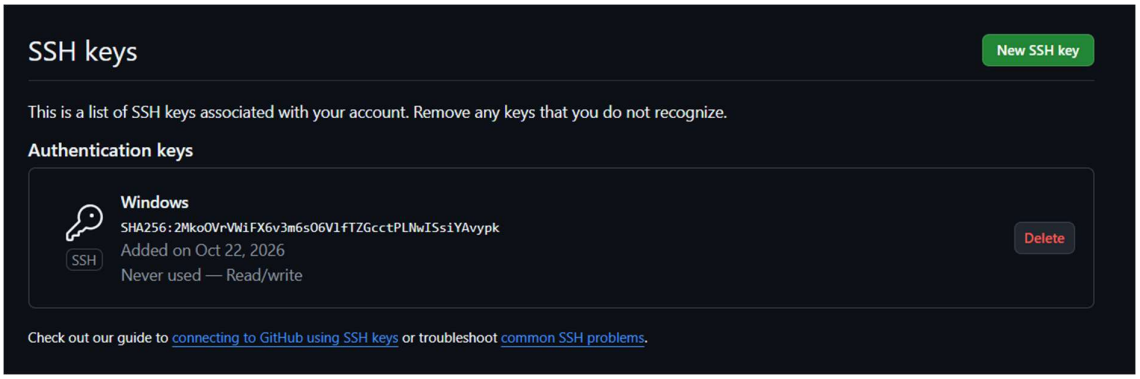
Step 3:- It will then ask passphrase just leave it blank. Now we get a path for ssh key file. Use cat to view the content

Step 4:- After using cat sshkey file copy the text from this file.

Step 5:- Now go to ssh key in settings of GitHub and create a new key give it a title and paste the key.

Step 6:- Now the ssh-keygen is configured.





18. Configure webhooks to github.

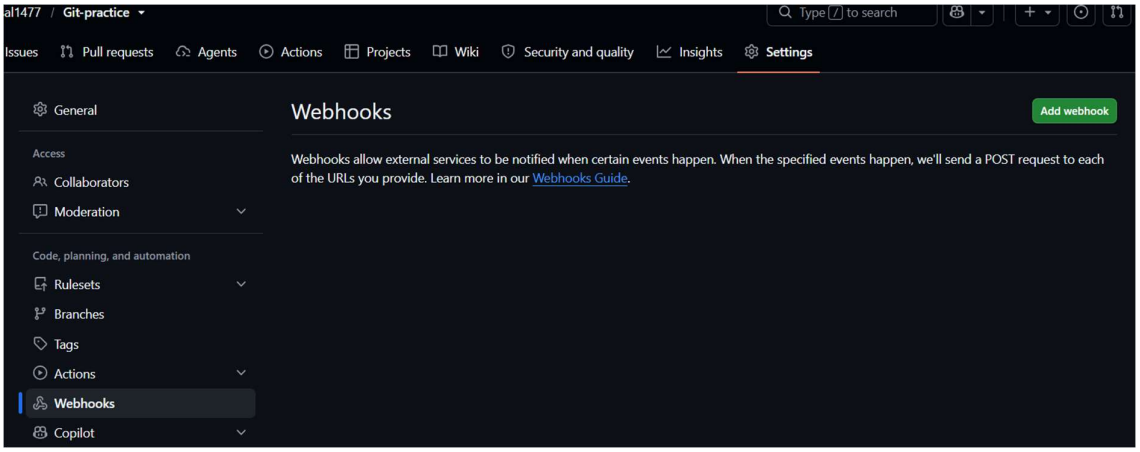
Step 1:- Configure webhooks to github.

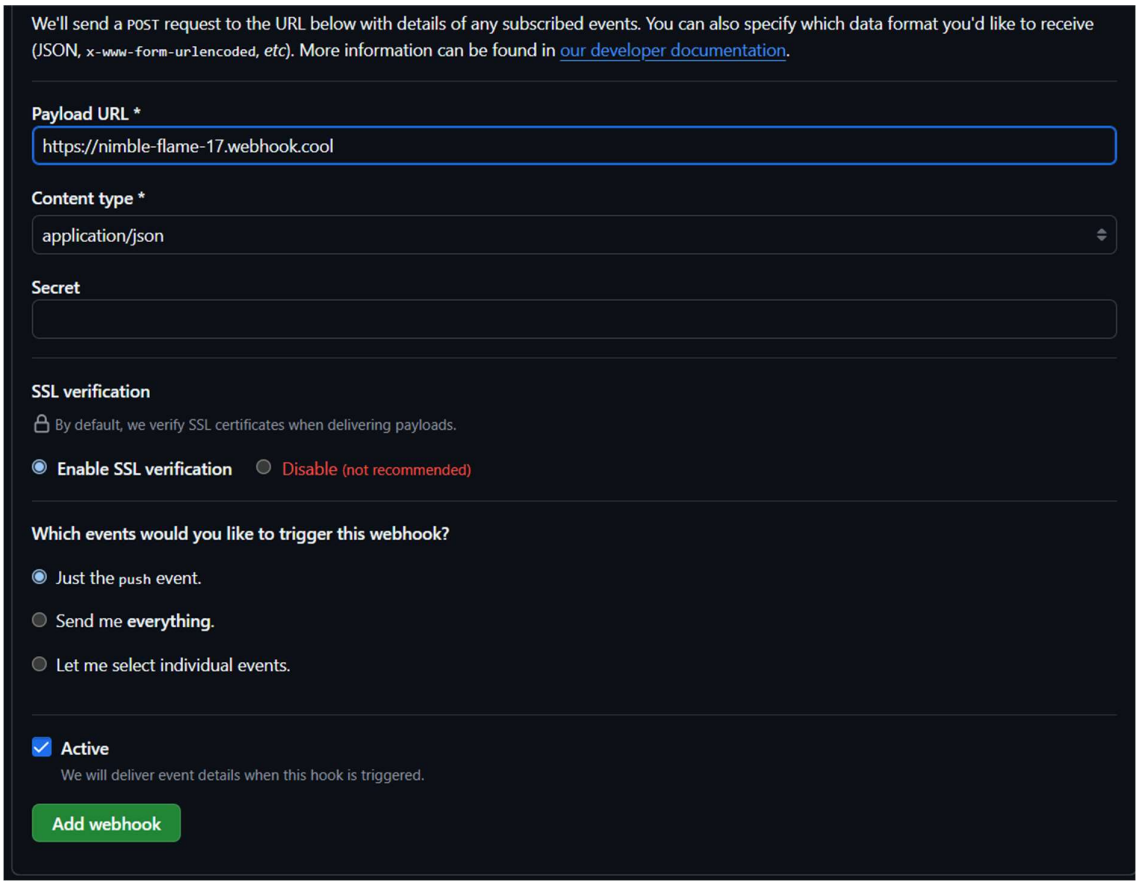
Step 2:- Open your GitHub repository.

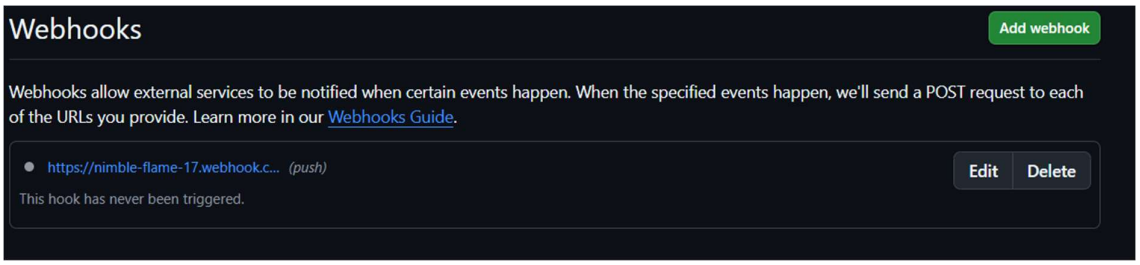
Step 3:- Go to Settings → Webhooks → Add webhook.

Step 4:- Get a webhook URL from official website and paste it in the Payload URL field.

Step 5:- Select application/json. Choose Just the push event







19. Basic understanding of .git file.

The .git directory is the core of a local Git repository.

If you delete it, your project files remain, but the repository's Git history and metadata are lost.

Important contents File/Directory Purpose HEAD Identifies the current branch or commit reference.

config Stores repository-specific Git settings.

objects/ Stores Git objects, including file contents and commits.

refs/ Stores references to branches and tags.

hooks/ Contains sample or configured scripts that run during Git events.
index Stores the staging area's information.

```
Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Tasks (main)
$ cd .git

Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Tasks/.git (GIT_DIR!)
$ ll
total 13
-rw-r--r-- 1 Faisal 197609  23 Oct 24 12:40 COMMIT_EDITMSG
-rw-r--r-- 1 Faisal 197609  21 Oct 24 12:35 HEAD
-rw-r--r-- 1 Faisal 197609 307 Oct 24 12:43 config
-rw-r--r-- 1 Faisal 197609  73 Oct 24 12:35 description
drwxr-xr-x 1 Faisal 197609   0 Oct 24 12:35 hooks/
-rw-r--r-- 1 Faisal 197609 649 Oct 24 12:40 index
drwxr-xr-x 1 Faisal 197609   0 Oct 24 12:35 info/
drwxr-xr-x 1 Faisal 197609   0 Oct 24 12:40 logs/
drwxr-xr-x 1 Faisal 197609   0 Oct 24 12:40 objects/
drwxr-xr-x 1 Faisal 197609   0 Oct 24 12:43 refs/

Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Tasks/.git (GIT_DIR!)
```

20. Check all the logs of git.

Step1:-To check every commit done in the project we can use git log

Step2:- The command get us full list of commits done with details.

Step3:-For detailed version we can use git log –stat. This show files added

```
Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Tasks/.git (GIT_DIR!)
$ git log
commit 8a17e1d0a428cf591c38d73fc10a755b36049f88 (HEAD -> main, origin/main)
Author: Faisal1477 <mohdfaisal1477@gmail.com>
Date:   Sat Oct 24 12:40:09 2026 +0530

    Initial Task Documents
```

```
Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Tasks/.git (GIT_DIR!)
$ git log --stat
commit 8a17e1d0a428cf591c38d73fc10a755b36049f88 (HEAD -> main, origin/main)
Author: Faisal1477 <mohdfaisal1477@gmail.com>
Date:   Sat Oct 24 12:40:09 2026 +0530

    Initial Task Documents

 Bash scripting 1.pdf | Bin 0 -> 1012883 bytes
 Bash_scripting 2.pdf | Bin 0 -> 3093420 bytes
 Basic linux.pdf      | Bin 0 -> 880136 bytes
 Git Task 1.pdf       | Bin 0 -> 1454006 bytes
 Linux Challenge.pdf  | Bin 0 -> 1483834 bytes
 Linux42 Tasks.pdf    | Bin 0 -> 5024620 bytes
 Server Tasks.pdf     | Bin 0 -> 3113123 bytes
 7 files changed, 0 insertions(+), 0 deletions(-)
```

21. Rename the commit message.

Step1:- First check if there is a commit present or not.

Step 2:- Then use git amend -m “Message” to rename the commit message.

Step 3:- Avoid using this because it changes the id in local not in remote repo. So evertime we change message git push should be done

```
Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Tasks (main)
$ git log
commit 43c242d234e3ad29b96c1e0dc109cd5c25041149 (HEAD -> main)
Author: Faisal1477 <mohdfaisal1477@gmail.com>
Date:   Sat Oct 24 12:40:09 2026 +0530

    Initial Task documents

Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Tasks (main)
$ git commit --amend -m "Linux, Bash, Git task documents uploaded"
[main fc96ab1] Linux, Bash, Git task documents uploaded
 Date: Sat Oct 24 12:40:09 2026 +0530
 7 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 Bash scripting 1.pdf
 create mode 100644 Bash_scripting 2.pdf
 create mode 100644 Basic linux.pdf
 create mode 100644 Git Task 1.pdf
 create mode 100644 Linux Challenge.pdf
 create mode 100644 Linux42 Tasks.pdf
 create mode 100644 Server Tasks.pdf

Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Tasks (main)
$ git log
commit fc96ab13f06993de374a55ce24cd0fb8455ecb46 (HEAD -> main)
Author: Faisal1477 <mohdfaisal1477@gmail.com>
Date:   Sat Oct 24 12:40:09 2026 +0530

    Linux, Bash, Git task documents uploaded
```

22. Merge multiple commits into single commit.

Step 1:- First check the logs, there should be more than two commits for this to work.

Step 2:- If there are more than two commits we can use git rebase -I Head~count (count is commit count from last to first)

Step 3:- This will take recent commit from the logs and open the vi editor

Step 4:- Now pick the base commit and change the other to squash so that they merge with base commit.

Step 5:- After saving this we see commit message in vi editor. If we to change we can or leave it after saving.

Step 6:- Now we can see the log to check the merged commits

```
Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Tasks (main)
$ git rebase -i HEAD~2
```

```
MINGW64:/d/Tasks
pick 9d2b155 Readme file is added
squash 0373ec1 Dummy file added

# Rebase fc96ab1..0373ec1 onto fc96ab1 (2 commands)
#
# Commands:
# p, pick <commit> = use commit
# r, reword <commit> = use commit, but edit the commit message
# e, edit <commit> = use commit, but stop for amending
# s, squash <commit> = use commit, but meld into previous commit
# f, fixup [-C | -c] <commit> = like "squash" but keep only the previous
#                    commit's log message, unless -C is used, in which case
#                    keep only this commit's message; -c is same as -C but
#                    opens the editor
# x, exec <command> = run command (the rest of the line) using shell
# b, break = stop here (continue rebase later with 'git rebase --continue')
# d, drop <commit> = remove commit
# l, label <label> = label current HEAD with a name
# t, reset <label> = reset HEAD to a label
# m, merge [-C <commit> | -c <commit>] <label> [# <oneline>]
```

```
MINGW64:/d/Tasks
# This is a combination of 2 commits.
# This is the 1st commit message:

Readme file is added &

# This is the commit message #2:

Dummy file added

# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# Date:      Tue Oct 27 18:05:06 2026 +0530
#
# interactive rebase in progress; onto fc96ab1
# Last commands done (2 commands done):
#    pick 9d2b155 Readme file is added
#    squash 0373ec1 Dummy file added
# No commands remaining.
# You are currently rebasing branch 'main' on 'fc96ab1'.
#
# Changes to be committed:
#       new file:   heyy
#       new file:   readme.md
```

```
Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Tasks (main)
[detached HEAD 9a4cb02] Readme file is added &
 Date: Tue Oct 27 18:05:06 2026 +0530
 2 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 heyy
 create mode 100644 readme.md
Successfully rebased and updated refs/heads/main.

Faisal@LAPTOP-4DO8SS4E MINGW64 /d/Tasks (main)
$ git log
commit 9a4cb024627c9d748f2d392e3e3b0c4a02268912 (HEAD -> main)
Author: Faisal1477 <mohdfaisal1477@gmail.com>
Date:   Tue Oct 27 18:05:06 2026 +0530

    Readme file is added &

    Dummy file added
```